

NAME – ASHRAF ALI

REG NO - 192524192

Q7. Consider the concept map: Recurrence Relation → Recursion Tree → Level Cost →

Total Cost → Complexity Construct the map showing how a recurrence expands into a

tree, how each level contributes a cost, and how summing across levels yields total

complexity. Justify how the hierarchical relationships in the map make the recursion tree

method an effective analysis tool.

Step-by-Step Analysis

1. Identify Parameters

For the recurrence relation describing the streaming algorithm:

$$T(n) = 2T(n/2) + n$$

The Master Theorem form is $T(n) = aT(n/b) + f(n)$:

- $a = 2$: The algorithm splits the processing job into 2 recursive subproblems (e.g., divide-and-conquer processing of user video streams).
- $b = 2$: The input data size n is halved at each stage.
- $f(n) = n$: The work done at the current level to divide and combine the streaming data.

2. Calculate the Critical Exponent

We evaluate n^a :

$$a = 2 = 1 \Rightarrow n^a = n^1 = n$$

3. Compare $f(n)$ with n^a and Justify the Case

Comparing $f(n) = n$ and $n^a = n$:

- Since $f(n) = \Theta(n^a) = \Theta(n)$, **Case 2 of the Master Theorem applies.**

- **Case 2 Formula:** When $f(n) = \Theta(n^a)$, the solution is:

$$T(n) = \Theta(n^a \log \log n) = \Theta(n \log \log n)$$

Python Source Code

```
import math

def analyze_recurrence(a, b, k):
    """
    Solves recurrences of the form  $T(n) = aT(n/b) + O(n^k)$ 
    For  $T(n) = 2T(n/2) + n$ ,  $k = 1$  since  $f(n) = n^1$ .
    """

    log_b_a = math.log(a, b)

    print("=" * 45)
    print("    MASTER THEOREM ANALYSIS RESULT    ")
    print("=" * 45)
    print(f"Recurrence:  $T(n) = \{a\}T(n/\{b\}) + n^{\{k\}}$ ")
    print("-----")
    print(f"Parameters identified:")
    print(f"  a (number of subproblems) = {a}")
    print(f"  b (subproblem size factor) = {b}")
    print(f"  f(n) (work per level)    =  $n^{\{k\}}$ ")
    print("-----")
    print(f"Critical Exponent:")
    print(f"   $\log_b(a) = \log_{\{b\}}(\{a\}) = \{\log\_b\_a:.1f\}$ ")
    print(f"   $n^{(\log\_b a)} = n^{\{\log\_b\_a:.1f\}}$ ")
    print("-----")
```

```
# Master Theorem Case Check
```

```
if log_b_a > k:
```

```
    case = "Case 1"
```

```
    complexity = f"Theta(n^{log_b_a:.1f})"
```

```
reason = f"n^{log_b_a} grows strictly faster than f(n)"
```

```
elif math.isclose(log_b_a, k):
```

```
    case = "Case 2"
```

```
    complexity = "Theta(n log n)"
```

```
    reason = f"f(n) matches n^{log_b_a} = n^{k}"
```

```
else:
```

```
    case = "Case 3"
```

```
    complexity = f"Theta(n^{k})"
```

```
    reason = f"f(n) grows strictly faster than n^{log_b_a}"
```

```
print(f"Result:")
```

```
print(f" Applied Case : {case} ({reason})")
```

```
print(f" Complexity : {complexity}")
```

```
print("=" * 45)
```

```
# Run analysis for T(n) = 2T(n/2) + n
```

```
analyze_recurrence(a=2, b=2, k=1)
```

```
main.py  [Icons]  Run  Output

1 import math
2
3 def analyze_recurrence(a, b, k):
4     """
5     Solves recurrences of the form T(n) = aT(n/b) + O(n^k)
6     For T(n) = 2T(n/2) + n, k = 1 since f(n) = n^1.
7     """
8     log_b_a = math.log(a, b)
9
10    print("=" * 45)
11    print("    MASTER THEOREM ANALYSIS RESULT    ")
12    print("=" * 45)
13    print(f"Recurrence: T(n) = {a}T(n/{b}) + n^{k}")
14    print("-----")
15    print(f"Parameters identified:")
16    print(f"  a (number of subproblems) = {a}")
17    print(f"  b (subproblem size factor) = {b}")
18    print(f"  f(n) (work per level)     = n^{k}")
19    print("-----")
20    print(f"Critical Exponent:")
21    print(f"  log_b(a) = log_{b}({a}) = {log_b_a:.1f}")
22    print(f"  n^{log_b_a} = n^{log_b_a:.1f}")
23    print("-----")
24
25    # Master Theorem Case Check
26    if log_b_a > k:
27        case = "Case 1"
28        complexity = f"Theta(n^{log_b_a:.1f})"
29        reason = f"n^{log_b_a} grows strictly faster than f(n)"
30    elif math.isclose(log_b_a, k):
31        case = "Case 2"
32        complexity = "Theta(n log n)"
33        reason = f"f(n) matches n^{log_b_a} = n^{k}"
34    else:
35        case = "Case 3"
36        complexity = f"Theta(n^{k})"
37        reason = f"f(n) grows strictly faster than n^{log_b_a}"
38
39    print(f"Result:")
40    print(f" Applied Case : {case} ({reason})")
41    print(f" Complexity : {complexity}")
42    print("=" * 45)
```

```
=====
MASTER THEOREM ANALYSIS RESULT
=====
Recurrence: T(n) = 2T(n/2) + n^1
=====
Parameters identified:
  a (number of subproblems) = 2
  b (subproblem size factor) = 2
  f(n) (work per level)     = n^1
=====
Critical Exponent:
  log_b(a) = log_2(2) = 1.0
  n^{log_b_a} = n^1.0
=====
Result:
  Applied Case : Case 2 (f(n) matches n^{log_b_a} = n^1)
  Complexity : Theta(n log n)
=====

=== Code Execution Successful ===
```

Q21. Consider the concept map: Sorting Algorithm → Merge Sort → Divide, Conquer, Combine → Recurrence Relation → $O(n \log n)$. Explain how Merge Sort is analyzed through its recursive structure and how each phase contributes to the

recurrence. Extend the map by linking it with Divide and Conquer as the parent strategy. Justify how the recurrence leads to the final asymptotic complexity. Interpret

how the completed concept map explains why Merge Sort is more efficient than

simple quadratic sorting methods.

Here is the complete application of the Master Theorem for the online video streaming platform's recurrence relation $T(n) = 2T(n/2) + n$, along with an updated Python script tailored to your code style.

Step-by-Step Mathematical Analysis

1. Identify Parameters

Given the recurrence relation for processing streaming data:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

We map it to the general Master Theorem form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$:

- $a = 2$: The platform splits data processing into 2 recursive subproblems (e.g., streaming chunks).
- $b = 2$: The user data size n is divided in half at each step.
- $f(n) = n$: The non-recursive work required to divide the workload and merge user stream data ($f(n) = \Theta(n^1)$, so $k = 1$).

2. Calculate the Critical Exponent

We evaluate n^a :

$$a = 2 = 1 \Rightarrow n^a = n^1 = n$$

3. Justify Case Selection & Determine Time Complexity

We compare $f(n) = n$ against $n^a = n$:

- Since $f(n) = \Theta(n^a) = \Theta(n)$, Case 2 of the Master Theorem applies.
- Case 2 Formula: $T(n) = \Theta(n^a \log \log n) = \Theta(n^1 \log \log n) = \Theta(n \log \log n)$.

PYTHON CODE

```
main.py  [Icons]  Share  Run  Output

1 import math
2
3 def master_theorem(a, b, k=1):
4     # Calculate log_b(a)
5     value = math.log(a, b)
6
7     print("Master Theorem Analysis")
8     print("-----")
9     print("a =", a)
10    print("b =", b)
11    print("f(n) = n^" + str(k))
12    print(f"n^(log_b a) = n^{value:.1f}")
13    print("-----")
14
15    # Case Evaluation
16    if value == k:
17        print("Case 2 applies [f(n) = Theta(n^(log_b a))]" )
18        print("Time Complexity = Theta(n log n)")
19    elif value > k:
20        print("Case 1 applies [f(n) = O(n^(log_b a - epsilon))]" )
21        print(f"Time Complexity = Theta(n^{value:.1f})")
22    else:
23        print("Case 3 applies [f(n) = Omega(n^(log_b a + epsilon))]" )
24        print(f"Time Complexity = Theta(n^{k})")
25
26 # Run analysis for T(n) = 2T(n/2) + n
27 master_theorem(2, 2, k=1)
```

```
Master Theorem Analysis
-----
a = 2
b = 2
f(n) = n^1
n^(log_b a) = n^1.0
-----
Case 2 applies [f(n) = Theta(n^(log_b a))]
Time Complexity = Theta(n log n)

--- Code Execution Successful ---
```